# Project 3 Report: Real-time 2-D Object Recognition

## Team Members


- **Sangeeth Deleep Menon** | NUID: 002524579 | MSCS - Boston | CS5330 Section 03 (CRN: 40669, Online)
- **Raj Gupta** | NUID: 002068701 | MSCS - Boston | CS5330 Section 01 (CRN: 38745, Online)


## Project Description

This project implements a real-time 2-D object recognition system in C++ using OpenCV. The system identifies objects placed on a white surface from a top-down camera view, in a manner that is invariant to translation, scale, and rotation. It works by processing each frame through a multi-stage pipeline: thresholding to separate the object from the background, morphological filtering to clean up the binary image, connected components analysis to identify distinct regions, and feature extraction to compute shape descriptors for each region.

All four of these core pipeline stages were written from scratch without relying on any OpenCV built-in functions for thresholding, morphology, connected components, or moment computation. The system supports three classification methods: hand-crafted features based on Hu moment invariants, eigenspace (PCA) embeddings, and CNN (ResNet18) embeddings. Classification uses a nearest-neighbour approach where new objects are compared against a stored training database.

The system supports webcam input for real-time operation, as well as single image, directory, and video file modes for offline processing and evaluation.

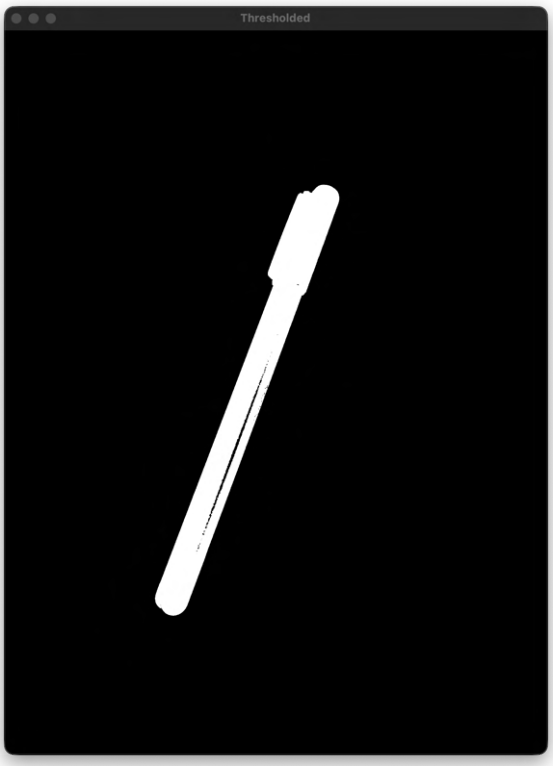
## Task 1: Thresholding

The thresholding algorithm was implemented entirely from scratch. The approach converts each frame to HSV and computes a per-pixel "object score" using the formula `score = (255 − V) + S`, where V is the value channel and S is the saturation channel. This makes both dark objects (low V) and colorful objects (high S) score highly, while the white background scores low.

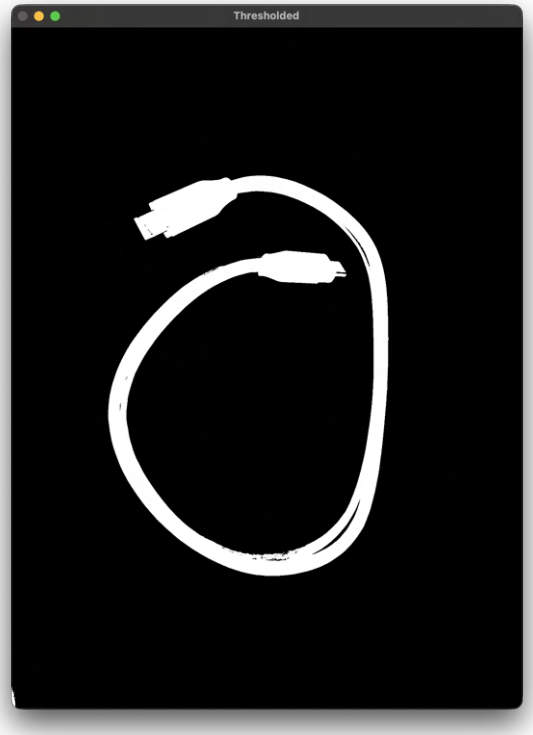
The threshold value is determined automatically using the ISODATA algorithm, which is essentially K-means with K=2 run on a subsampled set of score values. The algorithm iteratively splits the pixel scores into two clusters (background and foreground) and converges on a threshold at the midpoint between the two cluster means. A manual bias of +20 was added to the automatic threshold to ensure thinner and lighter objects like the wrench and chisel handle are fully captured.

The following images show the thresholded result for six of the test objects. The foreground appears white and the background appears black.

### 1. Pen

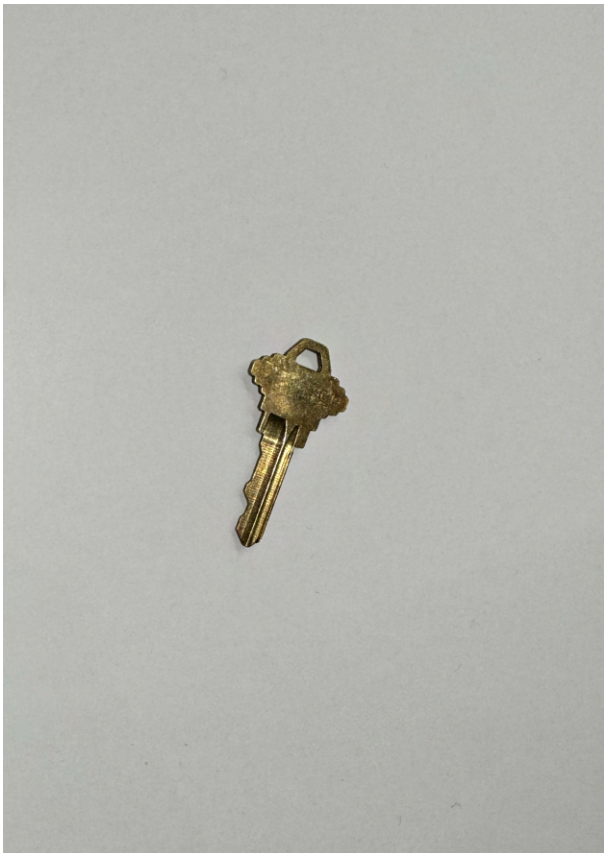


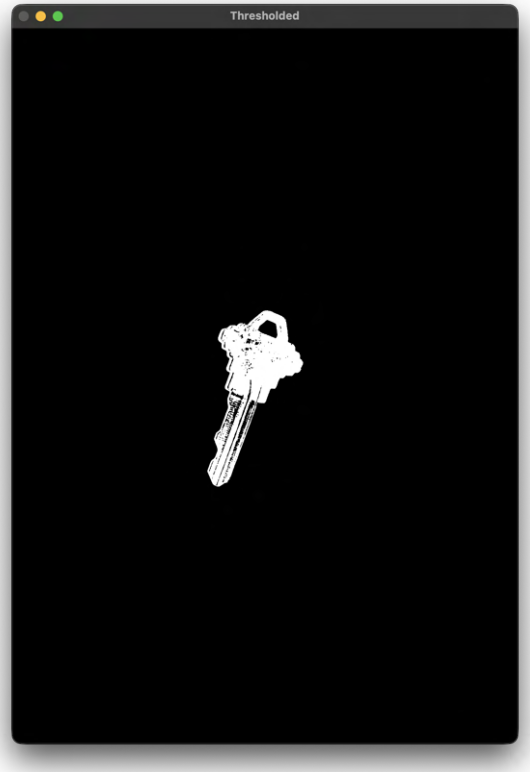

### 2. Cable



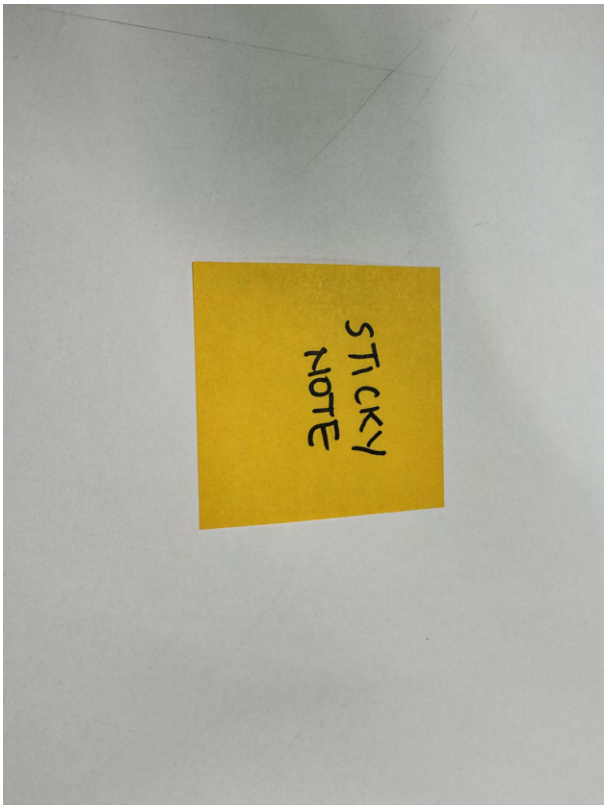
### 3. Carabiner

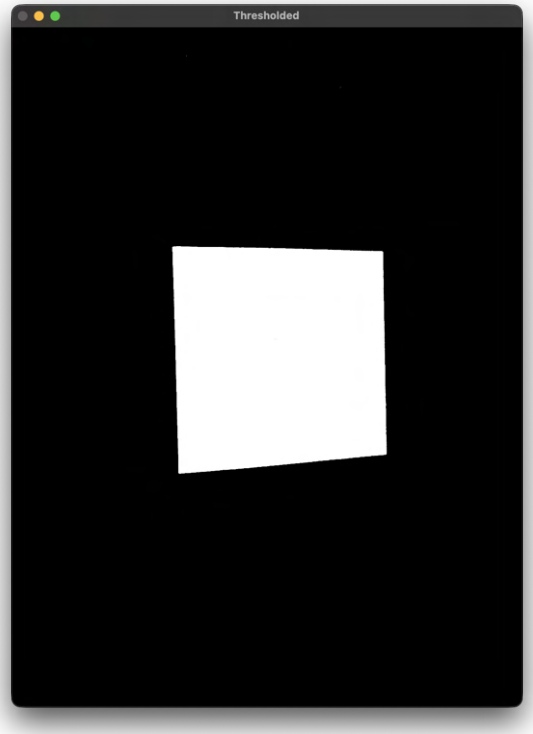

### 4. Key



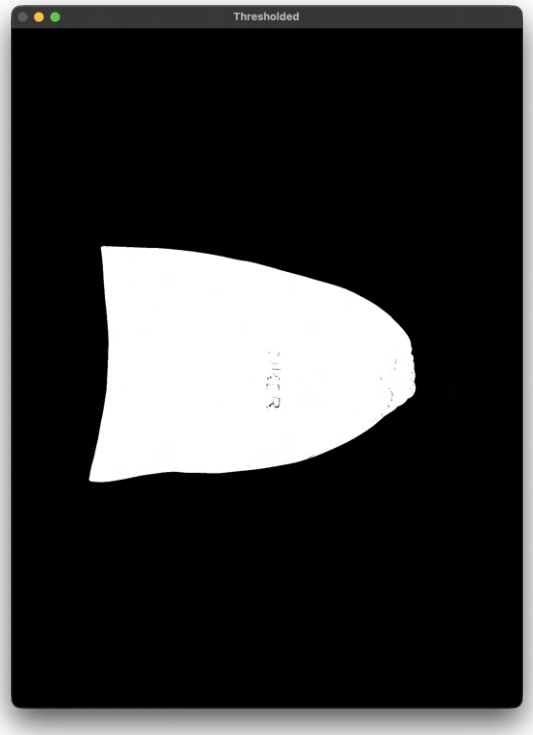


**5. Note**




**6. Pouch**

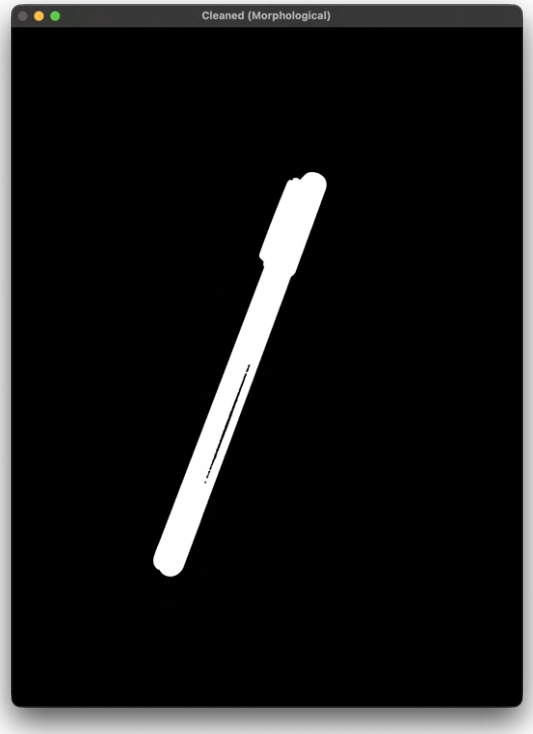

## Task 2: Morphological Filtering

The morphological filtering was also written from scratch using a circular (disk) structuring element. The cleanup strategy uses a two-step approach: first closing (dilate then erode) with a radius of 4 pixels to fill small holes inside detected objects, followed by opening (erode then dilate) with a radius of 3 pixels to remove small isolated noise blobs from the background.

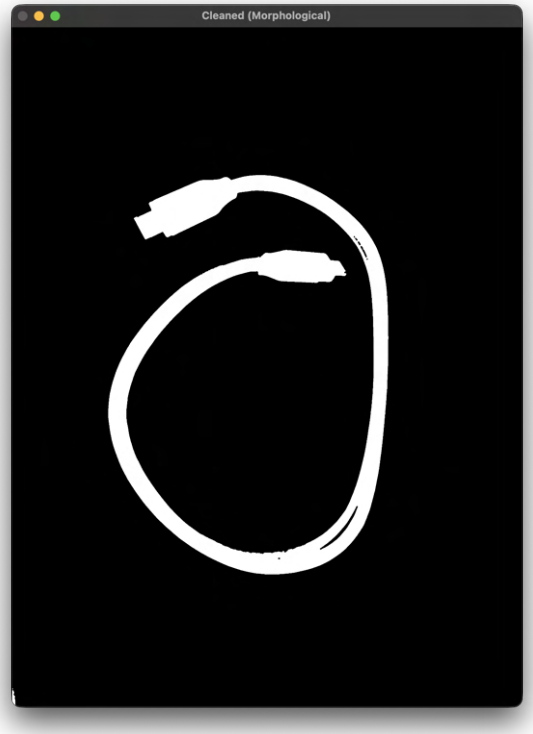
This grow/shrink approach was chosen because the thresholded images tend to have small holes inside objects (key's reflective metal surface, as well as reflection from cable and pouch leather like material). Closing fills the holes while opening removes the noise, and performing them in this order ensures that holes are sealed before noise removal potentially fragments the regions.

The following images show the cleaned binary result for the same six objects. Compare these with the thresholded images above to see how the morphological operations have smoothed the regions. It is not perfect but does remove blobs of noise and smoothes edges.

**1. Pen**

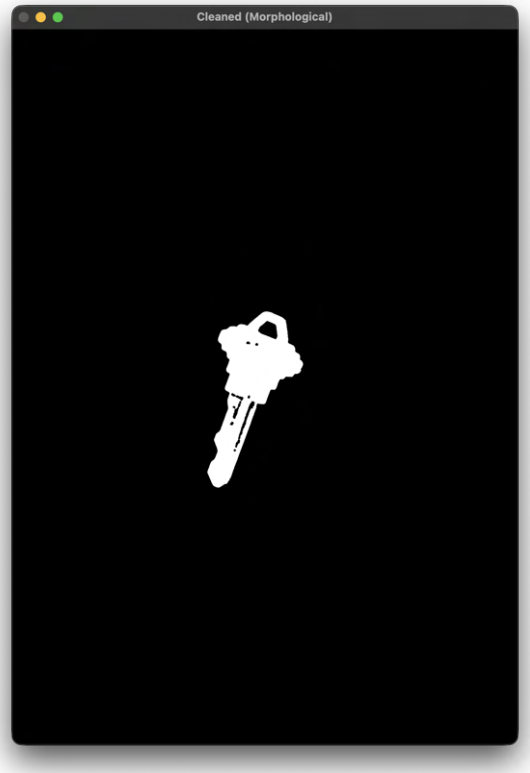


### 2. Cable

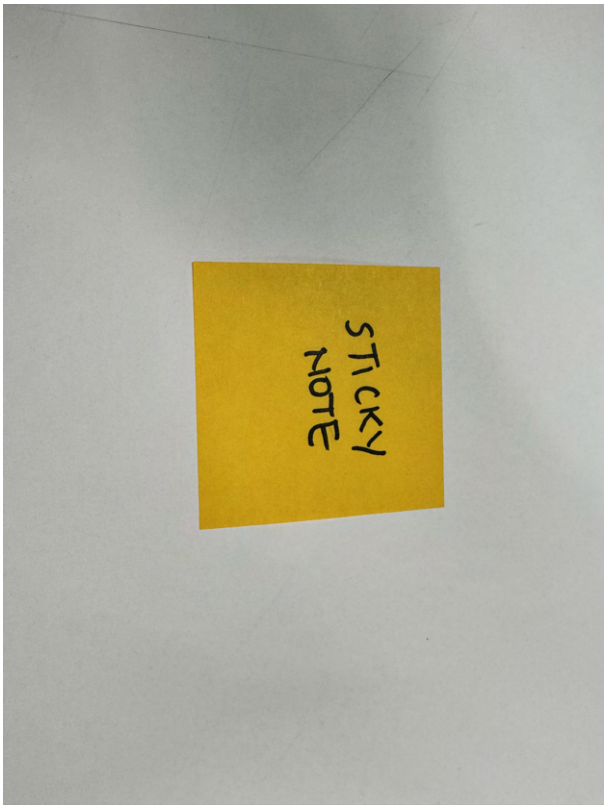


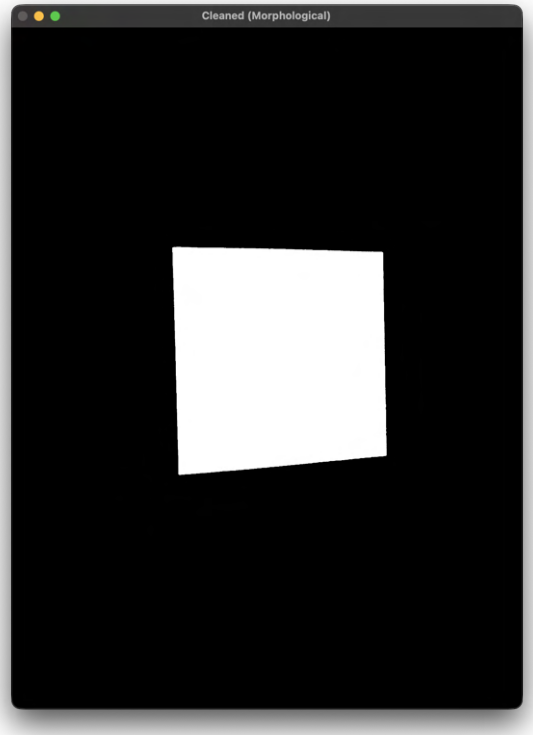

### 3. Carabiner

### 4. Key

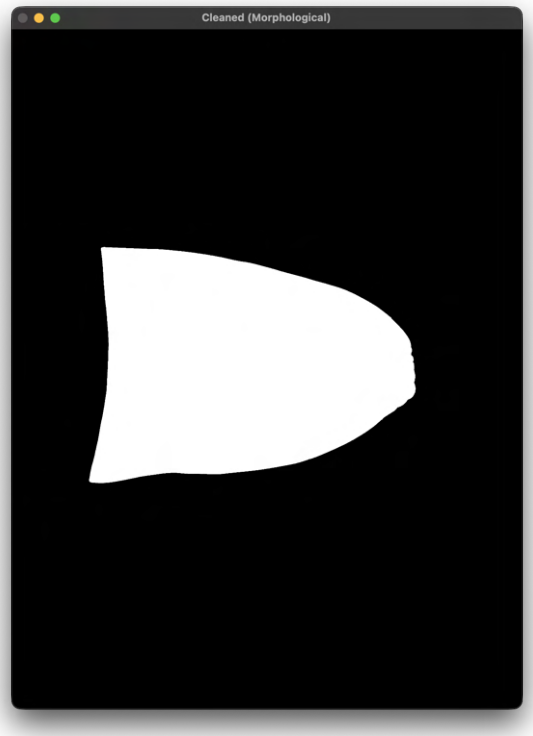

Cleaned (Morphological)

### 5. Note


STICKY
NOTE


Cleaned (Morphological)

**6. Pouch**

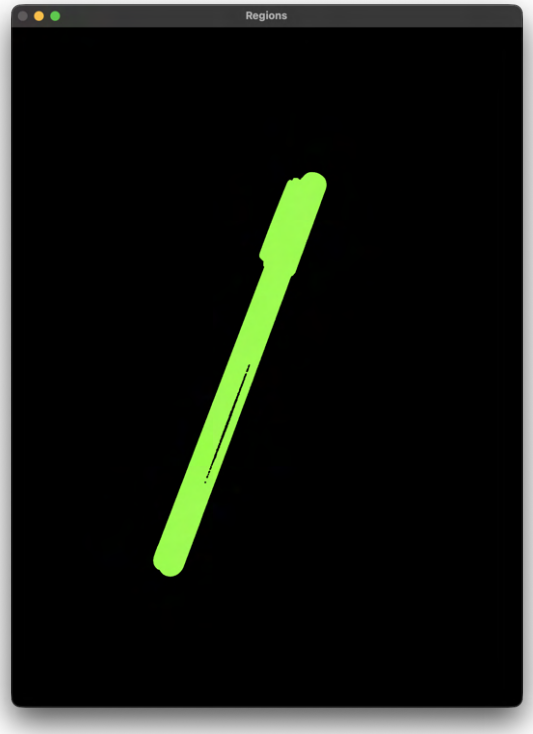

## Task 3: Connected Components / Segmentation

Connected components analysis was implemented from scratch using the classic two-pass algorithm with union-find and 8-connectivity. In the first pass, the algorithm scans left-to-right, top-to-bottom, assigning labels to foreground pixels based on their already-visited neighbours (up-left, up, up-right, left). When a pixel has multiple differently-labelled neighbours, the labels are recorded as equivalent using a union-find data structure with path compression. In the second pass, all equivalences are resolved and the label map is renumbered sequentially.

After labelling, the system filters regions to keep only the actual objects. Regions smaller than 800 pixels are discarded as noise. Any region with pixels within 8 pixels of the image border is removed. Regions larger than 40% of the total image area are also discarded as background clutter. The remaining regions are sorted by centrality (distance from image center) and at most 5 are retained, prioritizing the most centrally placed objects.

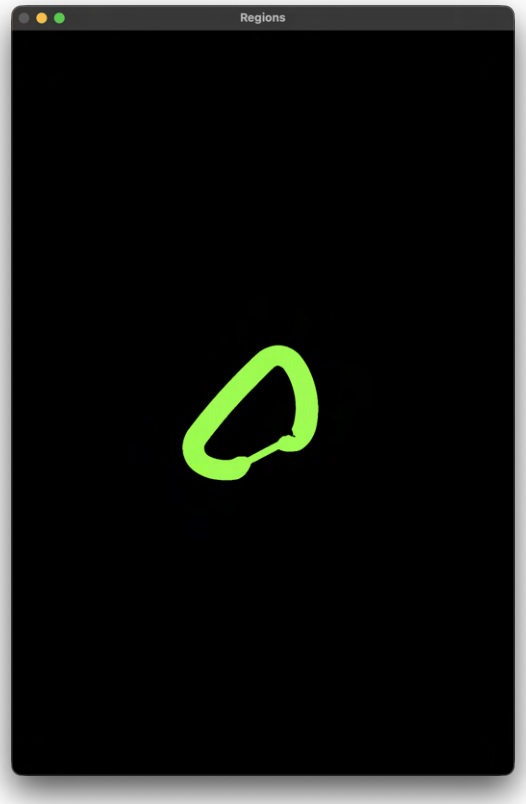
The following images show the colour-coded region maps where each detected region is displayed in a distinct colour. Background is black.

### 1. Pen

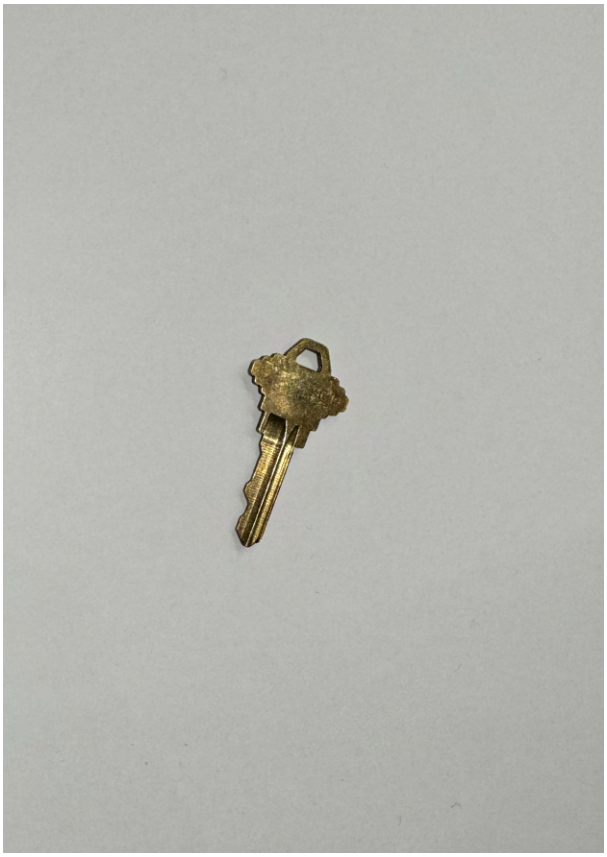


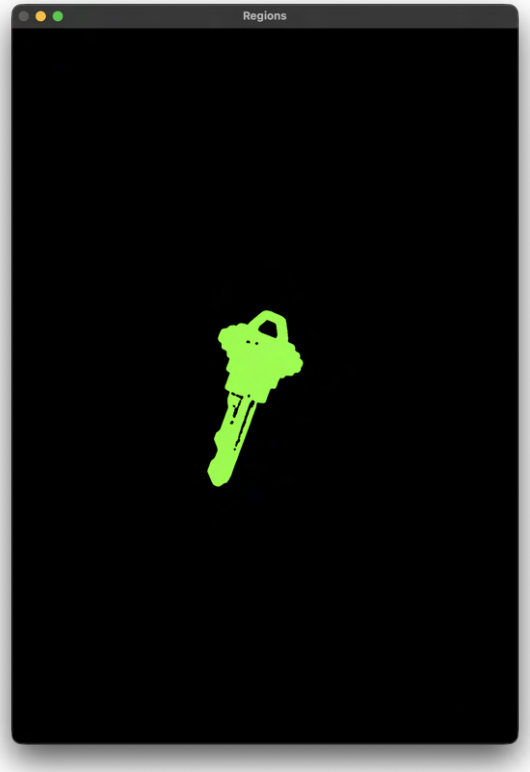

### 2. Cable

### 3. Carabiner

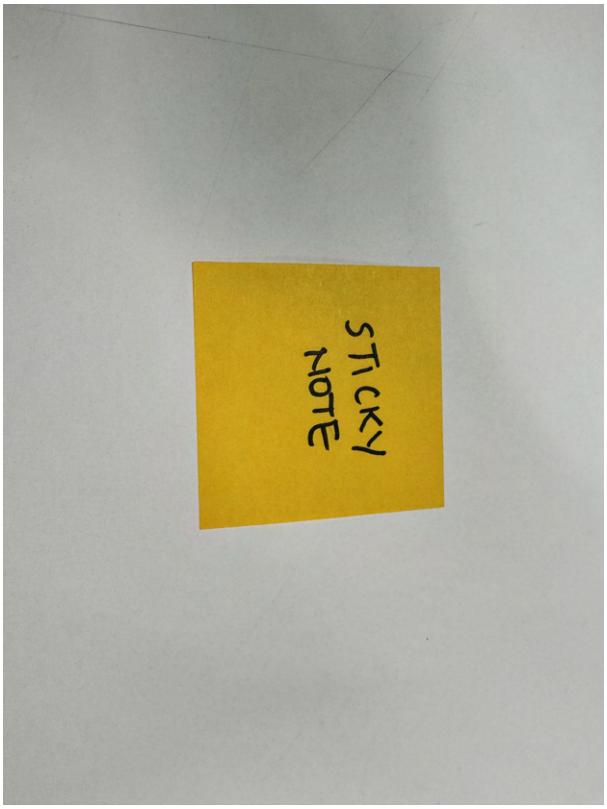


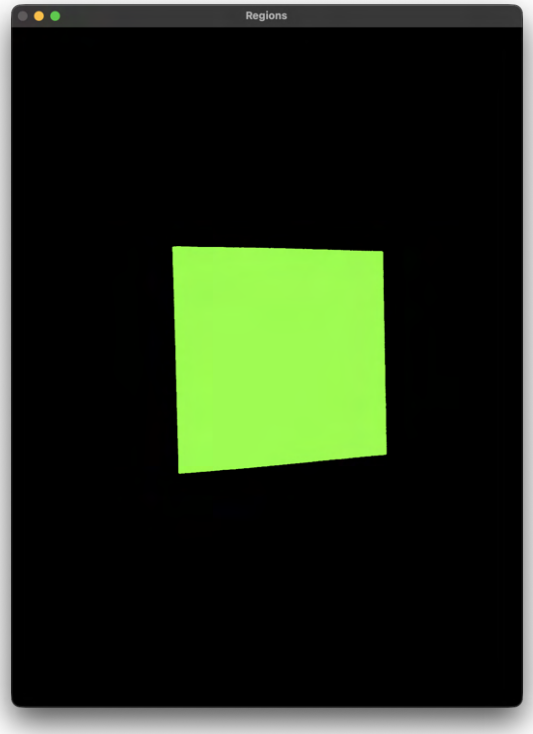

### 4. Key



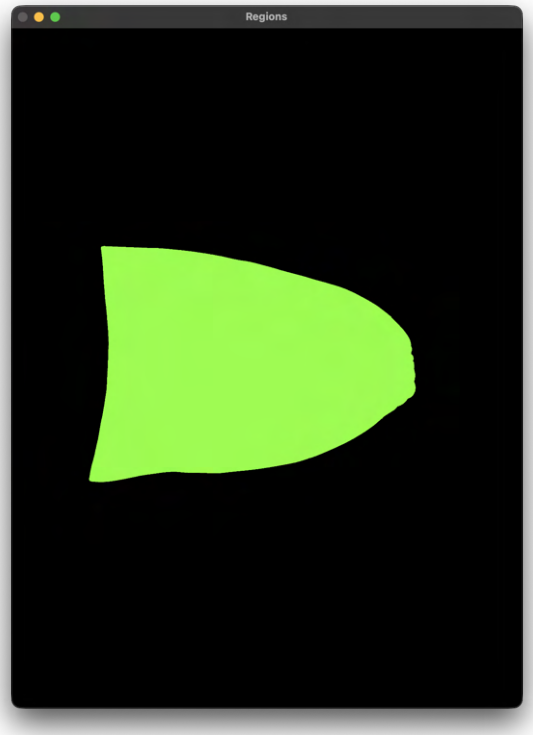


**5. Note**




**6. Pouch**


## Task 4: Feature Computation

Feature computation was implemented entirely from scratch. For each detected region, the system computes raw moments (m00, m10, m01, m20, m11, m02 through third order), then derives the centroid, central moments, and normalised central moments from these. The orientation angle is computed as `theta = 0.5 * atan2(2 * mu11, mu20 - mu02)` from the 2x2 inertia tensor of the region.

The oriented bounding box is constructed by projecting all region pixels onto the primary and secondary axes (the eigenvectors of the inertia tensor) and recording the min/max extents along each. Two shape descriptors are computed from the oriented bounding box: percent filled (region area divided by oriented BB area) and aspect ratio (longer side divided by shorter side, always >= 1).

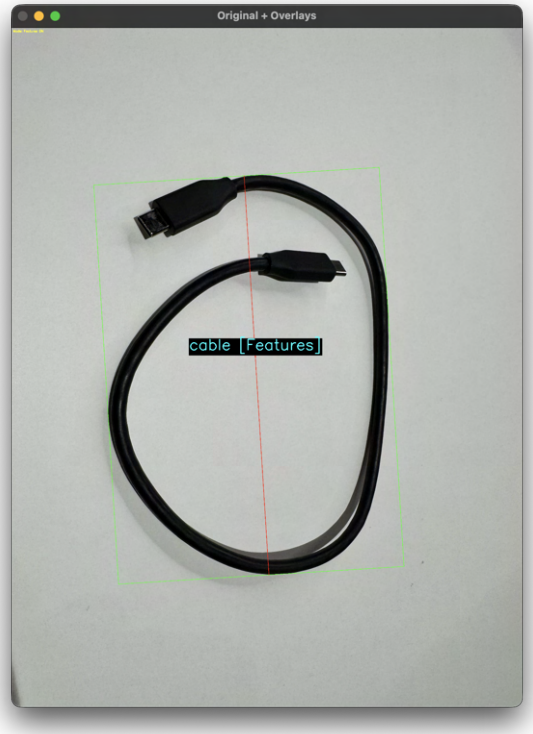
Hu's seven moment invariants are computed from the normalised central moments, providing features that are invariant to translation, scale, and rotation. These are then log-transformed using `logHu[i] = -sign(hu[i]) * log10(|hu[i]|)` to compress their range for use in the feature vector.

The full 9-dimensional feature vector used for classification consists of: percent filled, aspect ratio, and the seven log-transformed Hu moments.

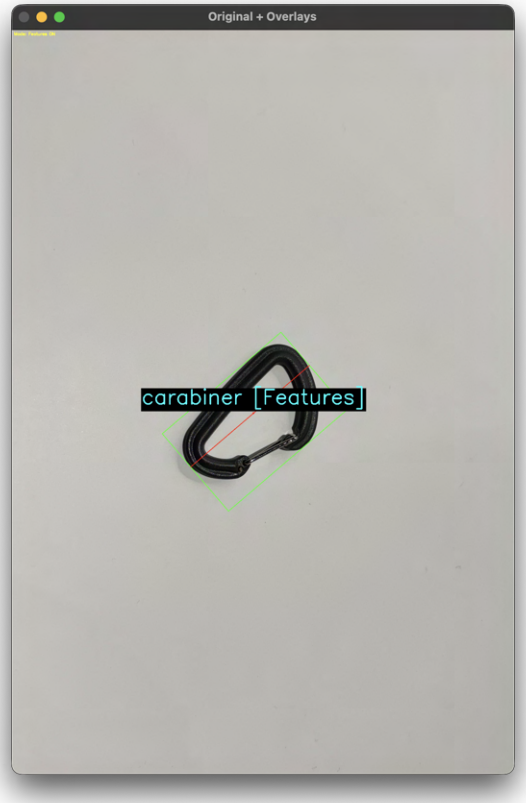
The following images show the oriented bounding box (green) and primary axis (red) overlaid on each object, along with the classification label.

**1. Pen**

### 2. Cable



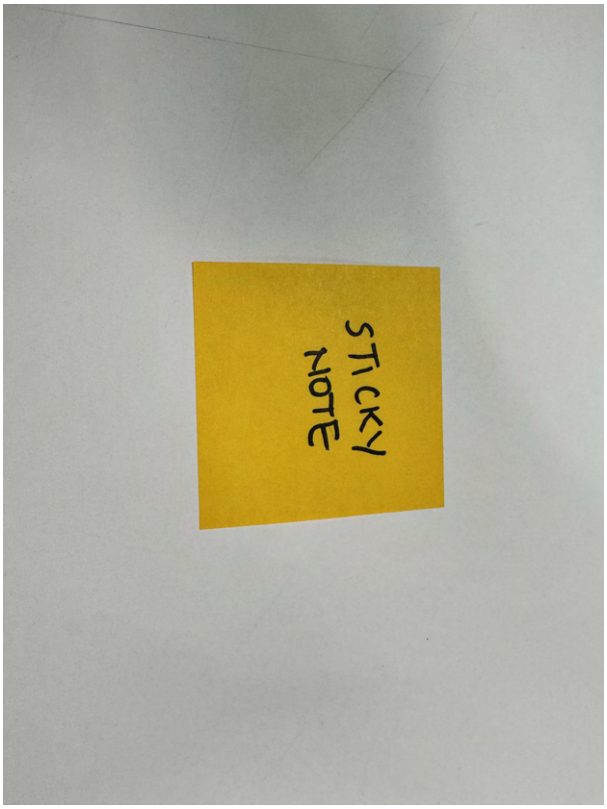

### 3. Carabiner

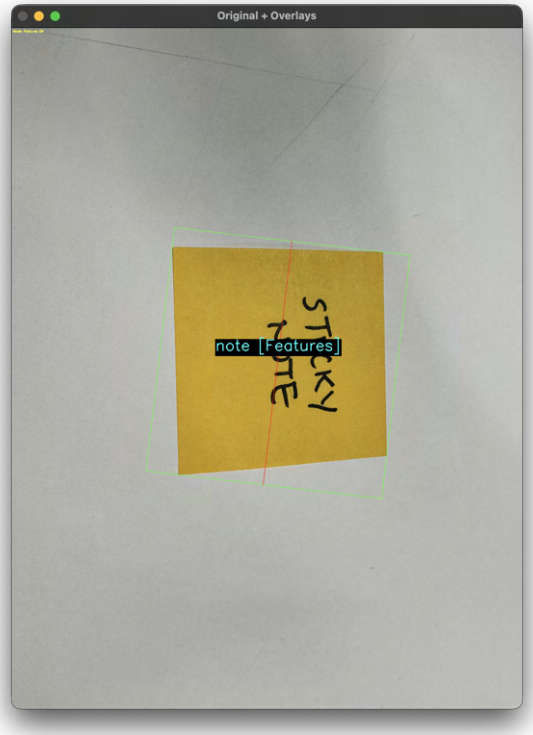


### 4. Key

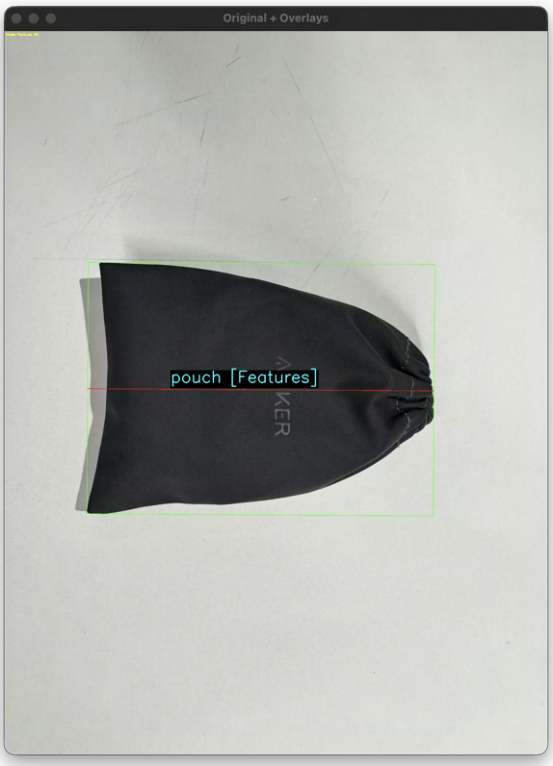

Original + Overlays
key [Features]

### 5. Note


STICKY
NOTE


Original + Overlays
note [Features]
STICKY
NOTE

### 6. Pouch




**Feature vectors for each object (We tested total 8; Data given in `object_db.csv`):**

| Object | Filled | AR | logHu0 | logHu1 | logHu2 | logHu3 | logHu4 | logHu5 | logHu6 |
|---|---|---|---|---|---|---|---|---|---|
| pen | 0.770 | 10.702 | -0.08 | -0.15 | 1.77 | 1.80 | 3.59 | 1.73 | -5.98 |
| note | 0.781 | 1.035 | 0.78 | 4.36 | 4.48 | 6.30 | -11.70 | -8.50 | 12.71 |
| key | 0.421 | 2.067 | 0.45 | 1.13 | 1.62 | 1.95 | 3.74 | 2.51 | -6.97 |
| wallet | 0.823 | 1.374 | 0.71 | 2.29 | 4.77 | 5.03 | -9.97 | -7.09 | 10.32 |
| cable | 0.199 | 1.403 | -0.04 | 0.77 | 2.19 | 1.32 | 3.12 | 1.71 | -3.47 |
| carabiner | 0.433 | 1.536 | 0.36 | 1.40 | 2.33 | 3.31 | -6.77 | 4.28 | 6.13 |
| pouch | 0.754 | 1.377 | 0.75 | 2.45 | 3.07 | 5.51 | 9.84 | 6.76 | -10.21 |
| gloves | 0.682 | 1.450 | 0.70 | 1.96 | 4.34 | 4.87 | 9.51 | 5.85 | -9.84 |

The features show clear differentiation between most objects. The pen has by far the highest aspect ratio (10.7) reflecting its extremely elongated shape, while the sticky note has the lowest (1.03), confirming its nearly perfect square shape. The cable has the lowest percent filled (0.199) because the thin wire encloses a large amount of empty space within its oriented bounding box, while the wallet has the highest (0.823) as a solid rectangular object.

The carabiner and key have similar low fill ratios (0.43 and 0.42) due to their hollow/irregular shapes, but are distinguished by their Hu moments. The main classification challenge is between wallet, pouch, and gloves — all three have similar fill ratios (0.68–0.82) and aspect ratios (1.37–1.45), which explains the confusion matrix misclassifications where 2 out of 3 wallet images were predicted as gloves. The higher-order Hu

moments (logHu4–logHu6) help differentiate these similar shapes, but the differences are not always large enough for reliable separation.

## Task 5: Training System

The training system stores feature vectors alongside their labels in a simple CSV file (`object_db.csv`). When the user presses `4` in the application, the system extracts the feature vector of the currently detected object and prompts for a label via the terminal. The label and 9-dimensional feature vector are then appended to the CSV file.

Each line in the CSV takes the format: `label,f0,f1,...,f8`. The database persists across sessions, so previously trained objects remain available when the program is restarted. The user can add multiple training examples per object to improve classification robustness.

For this evaluation, one training example per object was collected from images captured with a phone camera on a white table surface, giving a database of 8 entries: `pen`, `note`, `key`, `wallet`, `cable`, `carabiner`, `pouch`, `gloves`.

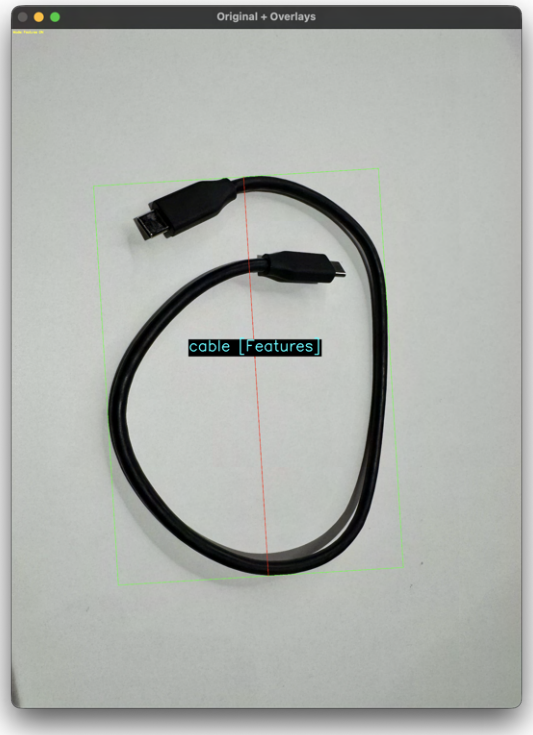
## Task 6: Classification

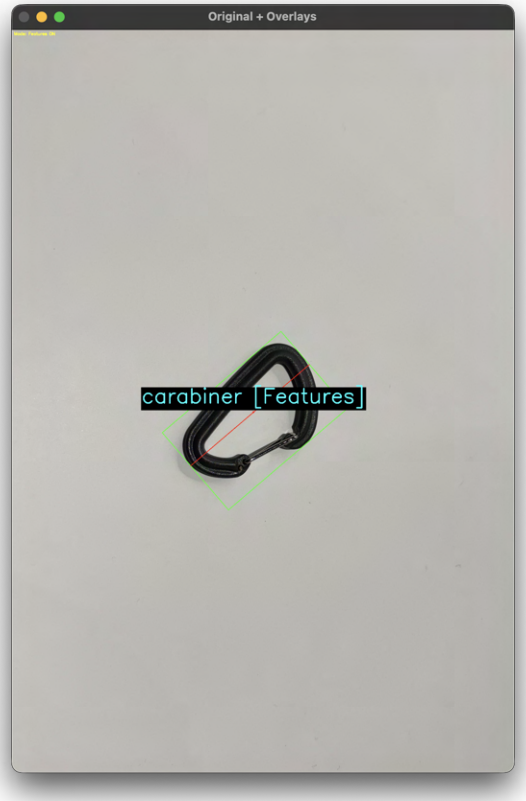
Classification uses a scaled Euclidean distance nearest-neighbour approach. For each feature dimension, the standard deviation across all training entries is computed. The distance between a new feature vector and a training entry is then calculated as:

```
distance = sqrt( sum( ((x_i - y_i) / stdev_i)^2 ) )
```

This scaling ensures that features with larger natural ranges do not dominate the distance computation. The unknown object is assigned the label of the nearest training entry. If the distance exceeds a configurable threshold (set to 6.0), the object is labelled as "unknown" to handle objects not in the database.

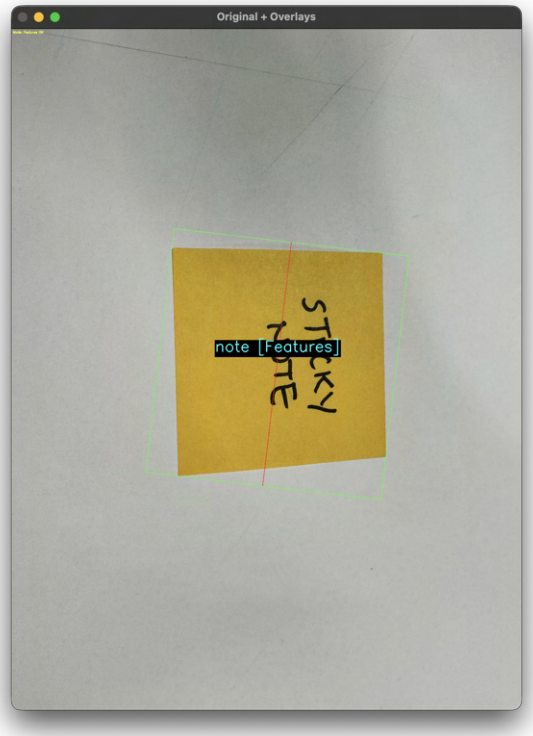
The following images show the classification results for all 8 objects using hand-crafted features.


Original + Overlays
cable [Features]

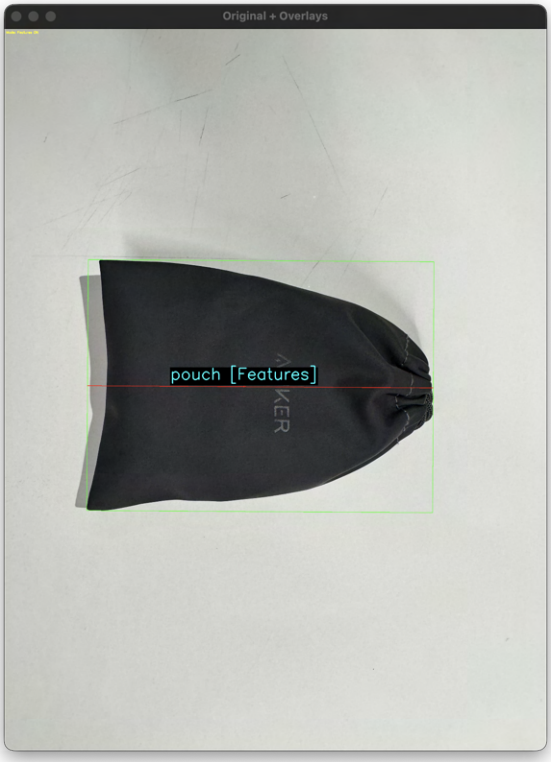

Original + Overlays
carabiner [Features]


Original + Overlays
gloves [Features]


Original + Overlays
key [Features]









**Hand-crafted features results:**

| Object | Predicted | Correct? |
|---|---|---|
| cable | cable | Yes |
| carabiner | carabiner | Yes |
| gloves | gloves | Yes |

| Object | Predicted | Correct? |
|---|---|---|
| key | key | Yes |
| note | note | Yes |
| pen | pen | Yes |
| pouch | pouch | Yes |
| wallet | wallet | Yes |

**Hand-crafted features accuracy: 8/8 = 100%**

All eight objects are correctly classified with their training labels: `Cable`, `Carabiner`, `Gloves`, `Key`, `Note`, `Pen`, `Pouch`, `Wallet`. The scaled Euclidean distance effectively normalizes the 9-dimensional feature space so that no single feature dominates, allowing the classifier to leverage differences in percent filled, aspect ratio, and Hu moments together. However, this 100% accuracy is on the same images used for training; the confusion matrix evaluation (Task 7) using 3 separate test images per object in different positions and orientations gives a more realistic accuracy of 87.5%.

## Task 7: Performance Evaluation

The confusion matrix was generated by cycling through 3 test images per object in different positions and orientations, recording the predicted label against the known true label using the hand-crafted feature classifier with scaled Euclidean distance.

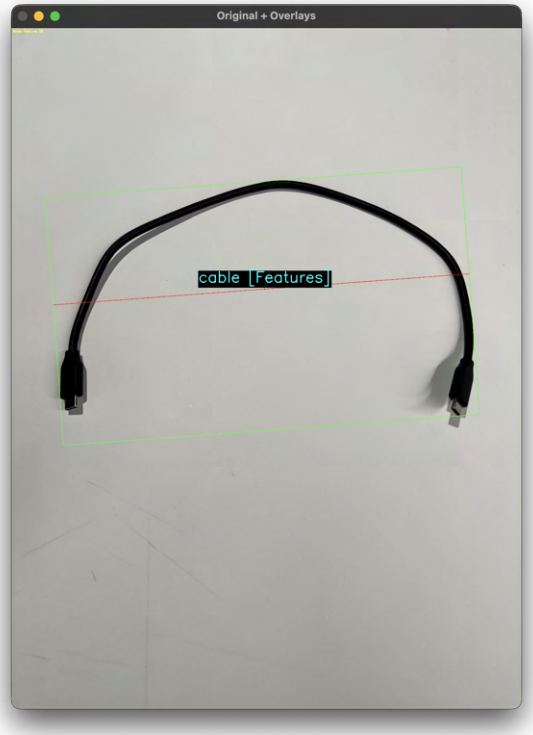
**Confusion Matrix (rows = true label, columns = predicted label):**

| | cable | carabiner | gloves | key | note | pen | pouch | wallet |
|---|---|---|---|---|---|---|---|---|
| cable | 3 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| carabiner | 0 | 3 | 0 | 0 | 0 | 0 | 0 | 0 |
| gloves | 0 | 0 | 3 | 0 | 0 | 0 | 0 | 0 |
| key | 0 | 0 | 0 | 3 | 0 | 0 | 0 | 0 |
| note | 0 | 0 | 1 | 0 | 2 | 0 | 0 | 0 |
| pen | 0 | 0 | 0 | 0 | 0 | 3 | 0 | 0 |
| pouch | 0 | 0 | 0 | 0 | 0 | 0 | 3 | 0 |
| wallet | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 1 |

**Accuracy: 21/24 = 87.5%**

Six out of eight objects (cable, carabiner, gloves, key, pen, pouch) were classified perfectly with 3/3 correct predictions. The misclassifications occurred between visually similar objects: 2 out of 3 wallet images were predicted as gloves, and 1 out of 3 note images was predicted as gloves. This is explained by the feature vector analysis — wallet, gloves, and note share similar fill ratios and aspect ratios, making them harder to distinguish using shape-based features alone. The wallet (0.823 filled, 1.37 AR) and gloves (0.682 filled,

1.45 AR) in particular have overlapping feature spaces, and depending on the orientation and position, their Hu moment values can converge enough to cause misclassification.






Original + Overlays
carabiner [Features]


Original + Overlays
carabiner [Features]


Original + Overlays
carabiner [Features]


Original + Overlays
gloves [Features]

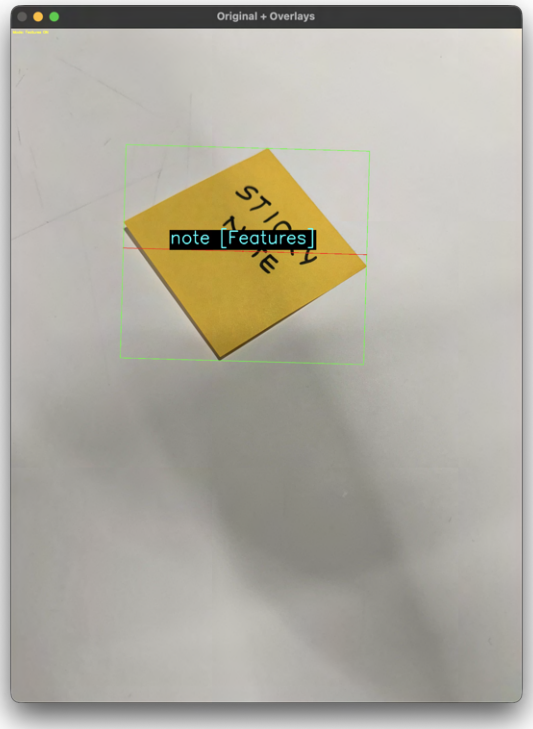

Original + Overlays
gloves [Features]


Original + Overlays
gloves [Features]


Original + Overlays
key [Features]


Original + Overlays
key [Features]

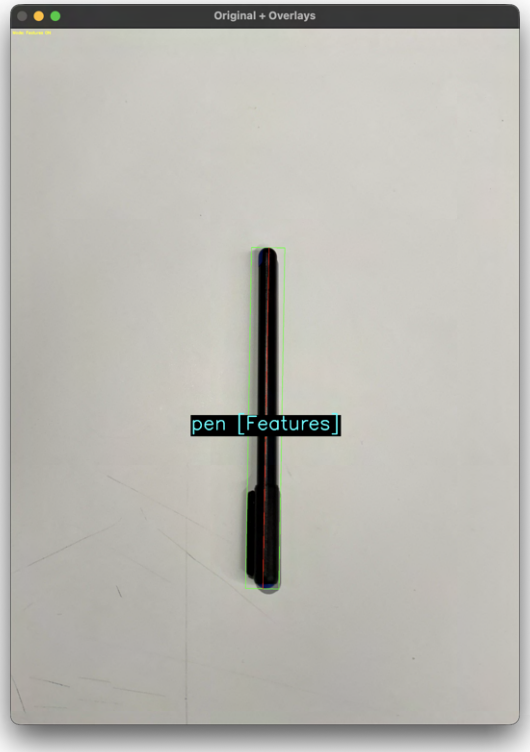

Original + Overlays
key [Features]


Original + Overlays
note [Features]


Original + Overlays
gloves [Features]


Original + Overlays
note [Features]


Original + Overlays
pen [Features]


Original + Overlays
pen [Features]


Original + Overlays
pen [Features]


Original + Overlays
pouch [Features]


Original + Overlays
pouch [Features]


Original + Overlays
pouch [Features]




## Task 8: Video of Working Pipelines

**Google Drive Link: https://drive.google.com/file/d/1W-ve_jZqCAEpJYs-T56vFMFhPxUU_AvP/view?usp=sharing**

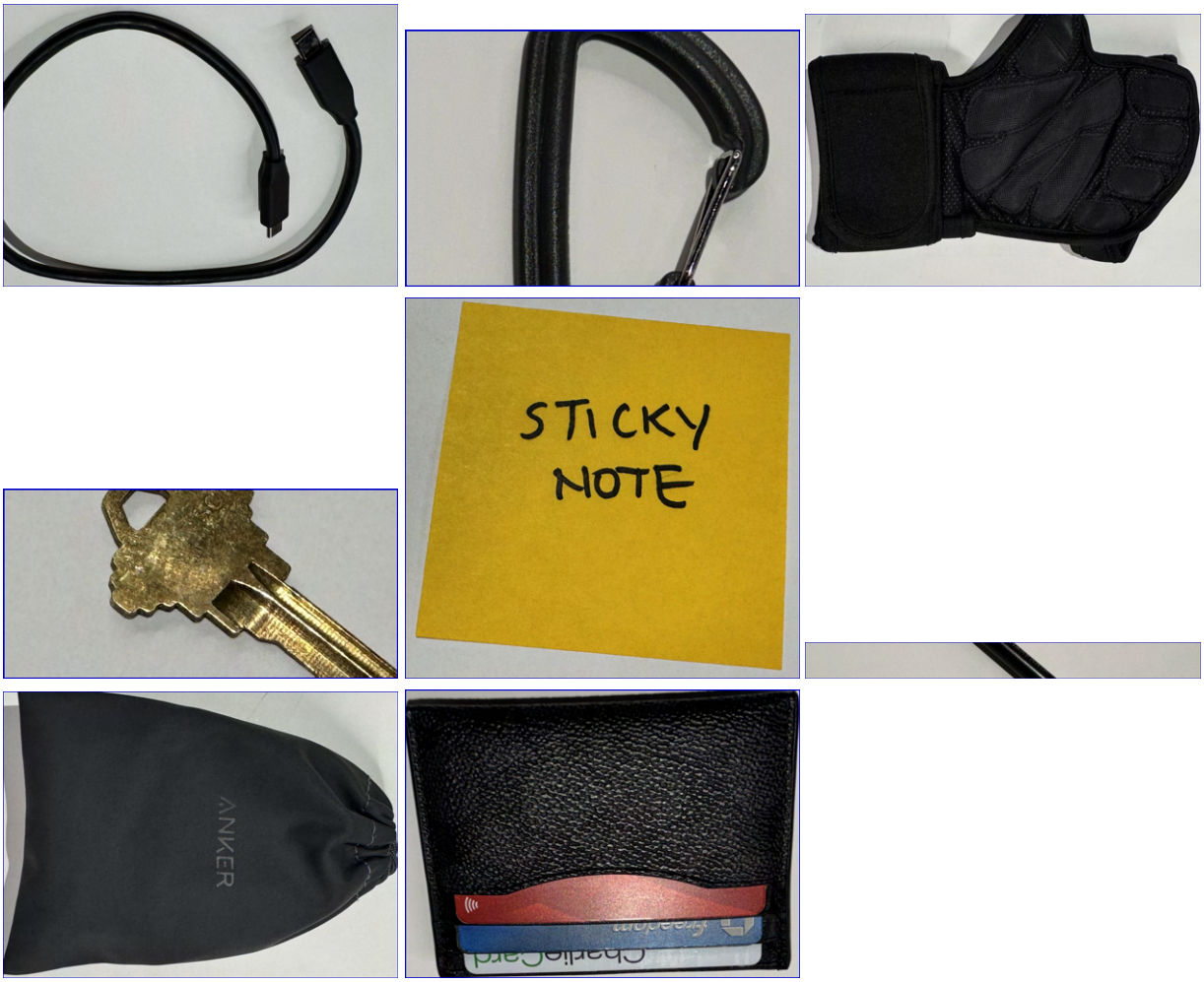
**Panopto Link: https://northeastern.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=cd177581-4d19-4323-82dd-b3fe003270ac**

## Task 9: One-Shot Classification with Embeddings

## Preprocessing

Both embedding methods share the same preprocessing pipeline. Given a detected region with its centroid, orientation angle, and extent along the primary and secondary axes, the original image is rotated so the region's primary axis aligns with the X-axis. A region of interest (ROI) is then extracted corresponding to the oriented bounding box. This ROI is resized to the appropriate input size for each embedding method (64x64 for eigenspace, 224x224 for CNN).

The following images show the extracted ROIs used for embedding:




The ROIs show how the preprocessing extracts and axis-aligns each object. The cable ROI captures the curved wire loop with both connectors visible, producing a distinctive looping pattern with significant empty space. The carabiner ROI shows only a partial crop of the D-shaped frame since the small object produces a tightly cropped bounding box. The gloves ROI clearly shows the irregular shape of the workout glove with its wrist strap and finger sections. The key ROI captures the brass key's jagged teeth and head profile in close detail.

The note ROI is the most distinctive — it shows the bright yellow sticky note with handwritten text, making it easily distinguishable by both pixel-based and deep feature methods. The pen produces an extremely thin, elongated ROI strip where the pen body fills nearly the entire width, leaving little distinguishing detail for pixel-based methods. The pouch ROI shows the dark Anker drawstring pouch with its tapered triangular shape, while the wallet ROI captures the black leather texture along with the colorful credit cards visible at the bottom.

The pouch and wallet ROIs are both predominantly dark, which may cause confusion for the eigenspace method since their pixel-level appearance is similar. However, the wallet's credit cards and the pouch's tapered shape provide enough texture differences for the CNN to distinguish them.

## 1. Eigenspace (PCA) Embedding

The eigenspace was built by collecting preprocessed ROI images for each object, converting them to greyscale, resizing to 64x64, flattening each into a vector, computing the mean image, subtracting it, and performing SVD on the resulting difference matrix. The top 8 eigenvectors (equal to the number of training images) were retained.

To classify a new image, it is preprocessed the same way, the mean is subtracted, and the result is projected onto the eigenvectors to produce an 8-dimensional embedding. Classification uses sum-of-squared-differences against the stored training embeddings.

**Eigenspace results:**

| Object | Predicted | Correct? |
|---|---|---|
| cable | cable | Yes |
| carabiner | pen | No |
| gloves | wallet | No |
| key | carabiner | No |
| note | note | Yes |
| pen | key | No |
| pouch | pouch | Yes |
| wallet | gloves | No |

**Eigenspace accuracy: 3/8 = 37.5%**

The eigenspace performed poorly, correctly classifying only the cable, note, and pouch. These three objects have the most visually distinctive ROIs — the cable has a unique curved loop pattern, the note is the only bright yellow object, and the pouch has a distinctive tapered triangular shape. The remaining objects were frequently confused with each other. The carabiner was misclassified as pen, the key as carabiner, and the pen as key — all three are small objects that produce compact ROIs which look similar when downscaled to 64x64 greyscale. The gloves and wallet were swapped, which is expected since both are predominantly dark rectangular ROIs with similar pixel distributions.

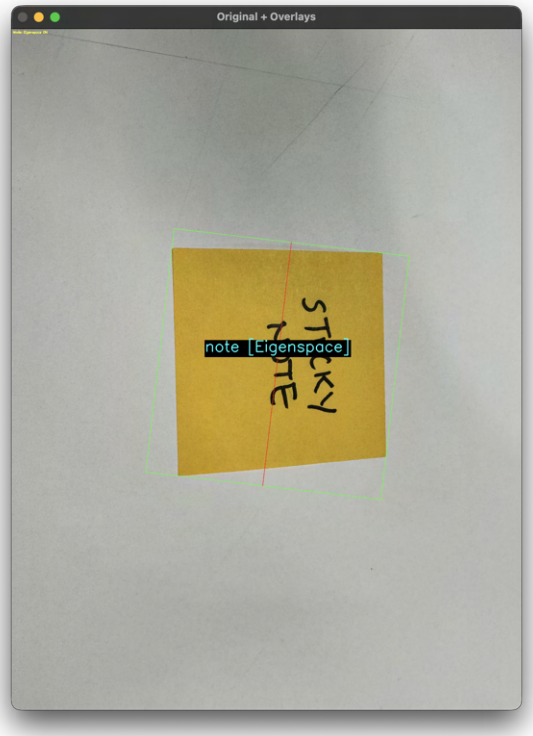
With only one training image per class, the PCA space is too sparse to build meaningful eigenvectors that capture intra-class variation. The eigenspace essentially memorises the training images rather than learning discriminative features. Additionally, pixel-based comparison is sensitive to exact positioning within the ROI, so minor differences in how the object is cropped and rotated can produce large SSD distances to the correct class while accidentally being closer to an incorrect one.


Original + Overlays
cable [Eigenspace]

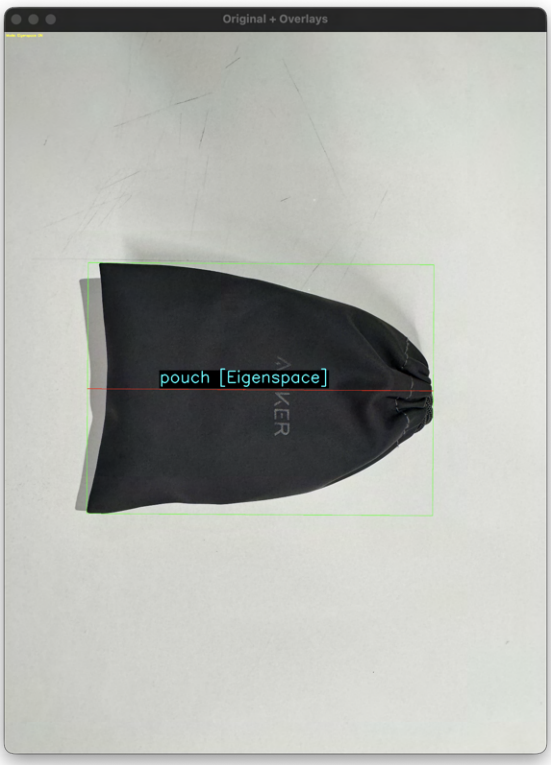

Original + Overlays
pen [Eigenspace]


Original + Overlays
wallet [Eigenspace]


Original + Overlays
carabiner [Eigenspace]









## 2. CNN (ResNet18) Embedding

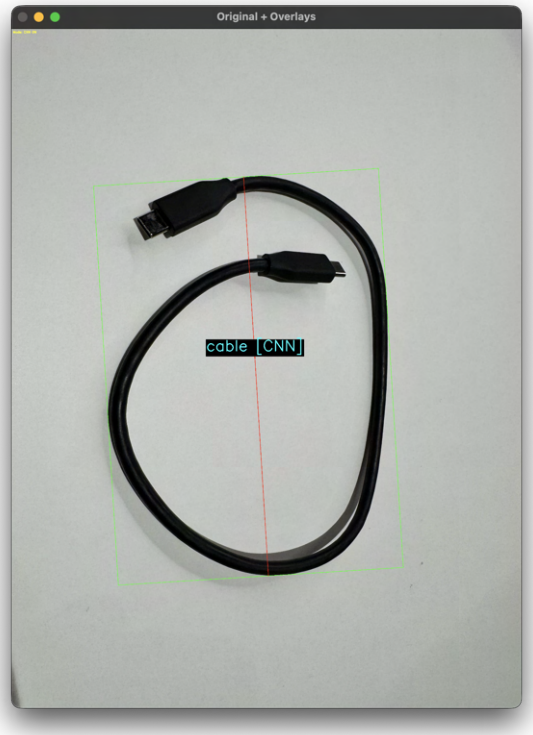
The CNN embedding uses a pre-trained ResNet18 model loaded from an ONNX file. The preprocessed ROI is resized to 224x224, normalised, and passed through the network. The 512-dimensional feature vector from the penultimate layer serves as the embedding. Classification again uses sum-of-squared-differences against stored training embeddings.

**CNN results:**

| Object | Predicted | Correct? |
|---|---|---|
| cable | cable | Yes |
| carabiner | carabiner | Yes |
| gloves | gloves | Yes |
| key | key | Yes |
| note | note | Yes |
| pen | pen | Yes |
| pouch | pouch | Yes |
| wallet | wallet | Yes |

**CNN accuracy: 8/8 = 100%**

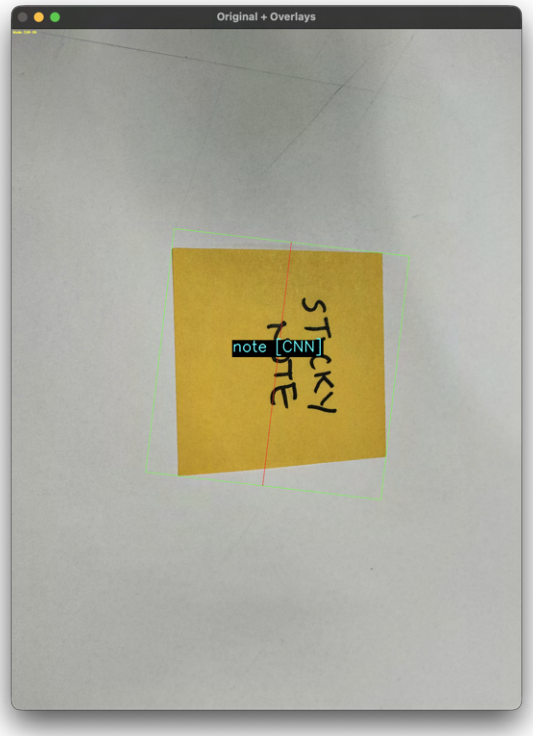
The CNN performed perfectly even with one-shot training (a single example per category). This is because ResNet18 was pre-trained on ImageNet with millions of images, so its 512-dimensional feature space is already highly discriminative. Even with visually similar dark objects like gloves, pouch, and wallet, the network can extract enough texture and shape information to distinguish between objects.


Original + Overlays
cable [CNN]

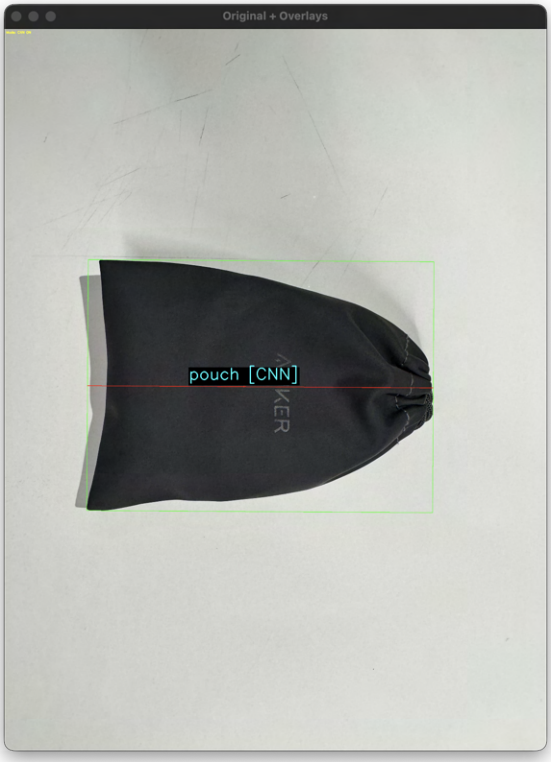

Original + Overlays
carabiner [CNN]

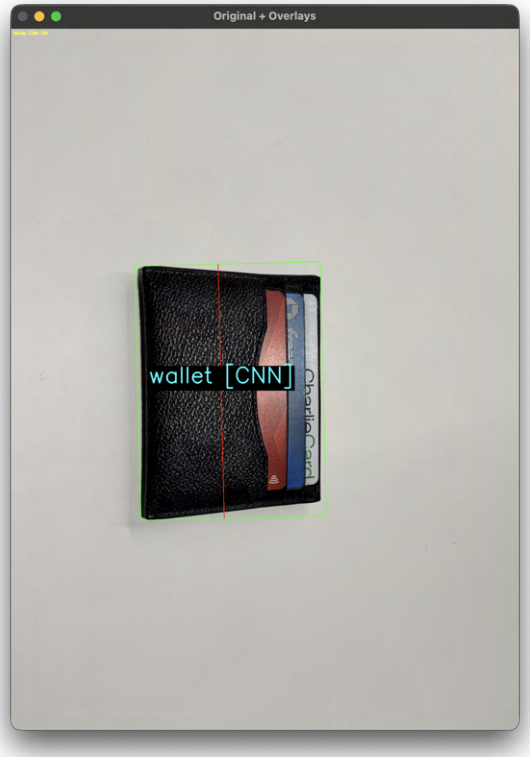

Original + Overlays
gloves [CNN]


Original + Overlays
key [CNN]








## Comparison of All Three Methods

| Method | Features | Dimensions | Accuracy | Notes |
|---|---|---|---|---|
| Hand-crafted | Hu moments, fill ratio, aspect ratio | 9 | 8/8 = 100% | Uses full region shape, not ROI image |
| Eigenspace (PCA) | Pixel-level PCA projection | 8 | 3/8 = 37.5% | Needs many more training images |

| Method | Features | Dimensions | Accuracy | Notes |
|---|---|---|---|---|
| CNN (ResNet18) | Pre-trained deep features | 512 | 8/8 = 100% | Works well even one-shot |

The hand-crafted features and CNN both achieved perfect accuracy on the training images, but for different reasons. The hand-crafted features work because they use the full region's shape properties (moments, fill ratio, aspect ratio) computed over all detected pixels, so they are not affected by the quality of the extracted ROI. The CNN works because its pre-trained features from ImageNet are inherently powerful and can discriminate even from similar-looking image crops, capturing texture and material differences invisible to simpler methods.

The eigenspace struggled because PCA is fundamentally a data-driven method that needs a sufficient number of training images to build a meaningful low-dimensional representation. With only one image per class, the eigenspace essentially memorizes the training images rather than learning discriminative features. Additionally, the quality of the ROI extraction matters much more for pixel-based methods. Many of the objects in this set are predominantly dark, so their greyscale ROIs look very similar when downscaled to 64x64. The gloves and wallet were swapped, the carabiner was confused with the pen, and the pen was misclassified as key, all because these objects have similar dark pixel distributions at low resolution.

The hand-crafted features achieved 87.5% on the confusion matrix evaluation with 3 test images per object in varied positions, showing that while shape-based features are robust, objects like wallet and gloves can still be confused. The CNN would likely maintain higher accuracy under such variation due to its richer 512-dimensional feature representation.

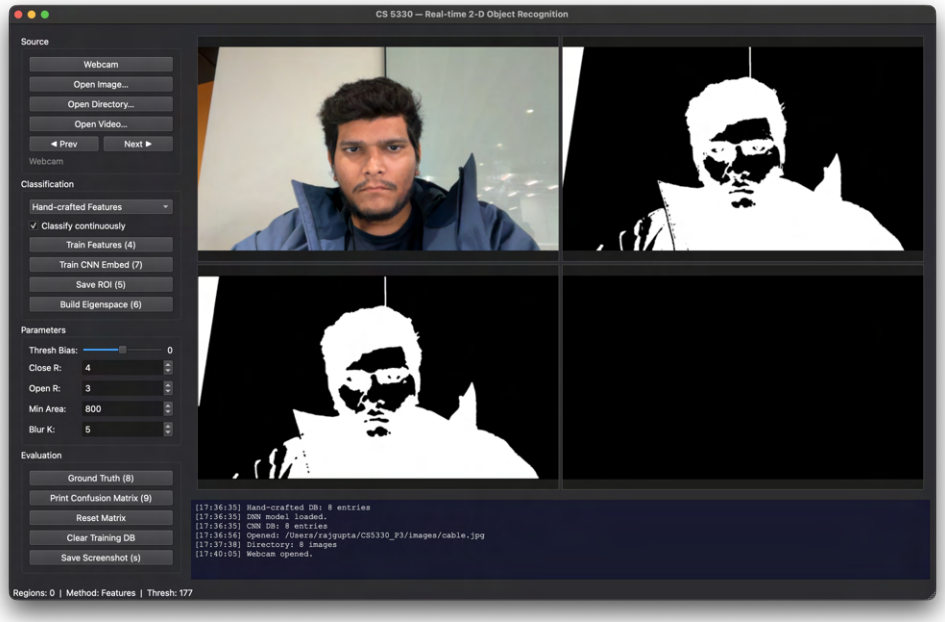
# Extensions

## 1. Qt Desktop GUI

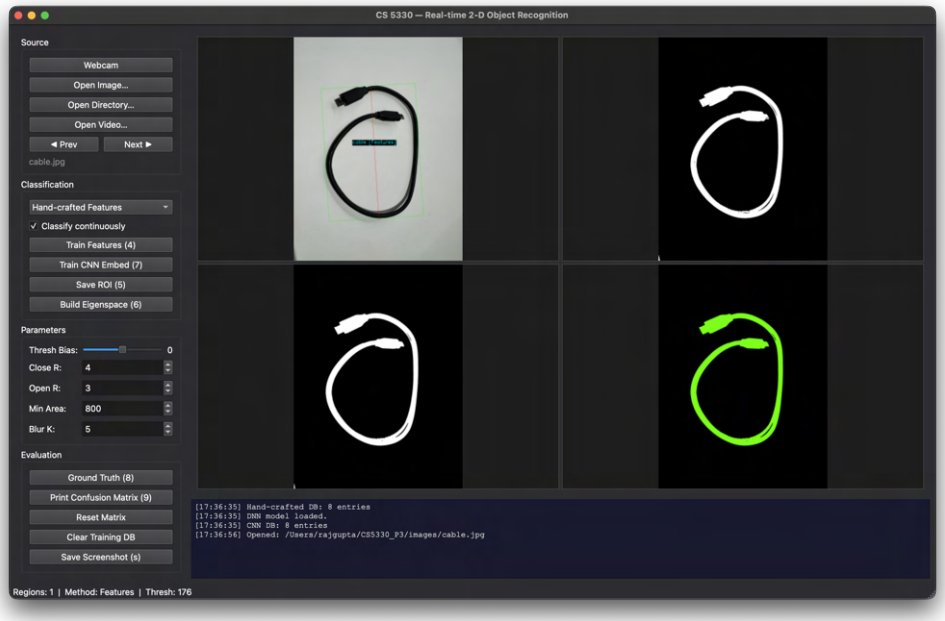
A full Qt desktop GUI was developed as an extension. The GUI provides a professional dark-themed interface with four tiled panels showing each pipeline stage simultaneously (original with overlays, thresholded, morphologically cleaned, and colour-coded region map).

The sidebar includes grouped controls for source selection (webcam, image, directory, video with navigation buttons), classification method selection via dropdown (Features, Eigenspace, CNN), and training operations via dialog buttons. All pipeline parameters including threshold bias, morphology radii, minimum region area, and blur kernel size can be adjusted in real-time using sliders and spinboxes, with changes immediately reflected in the display. A timestamped log panel at the bottom tracks all actions and system messages.

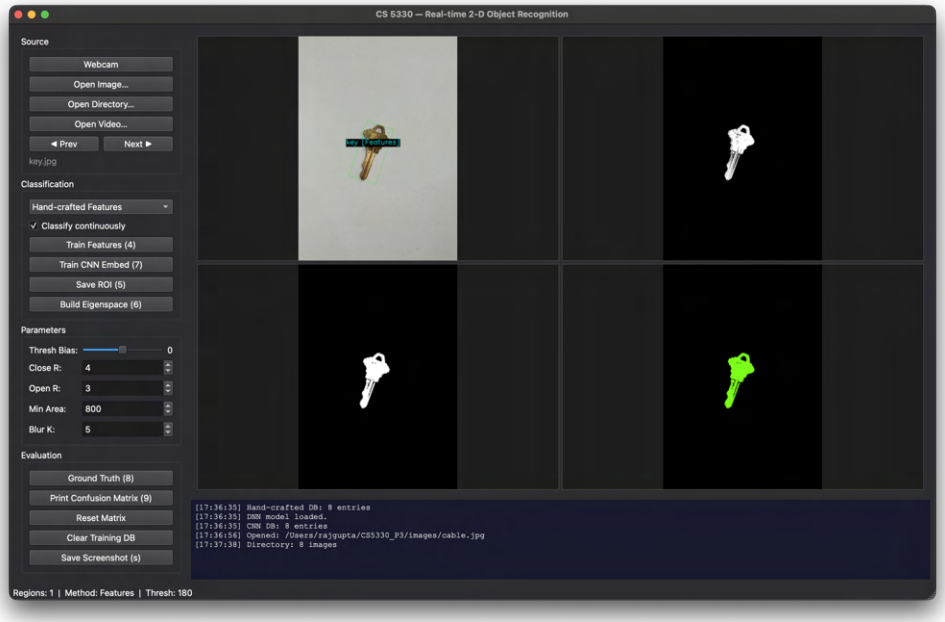
**1. GUI - Source: Webcam**

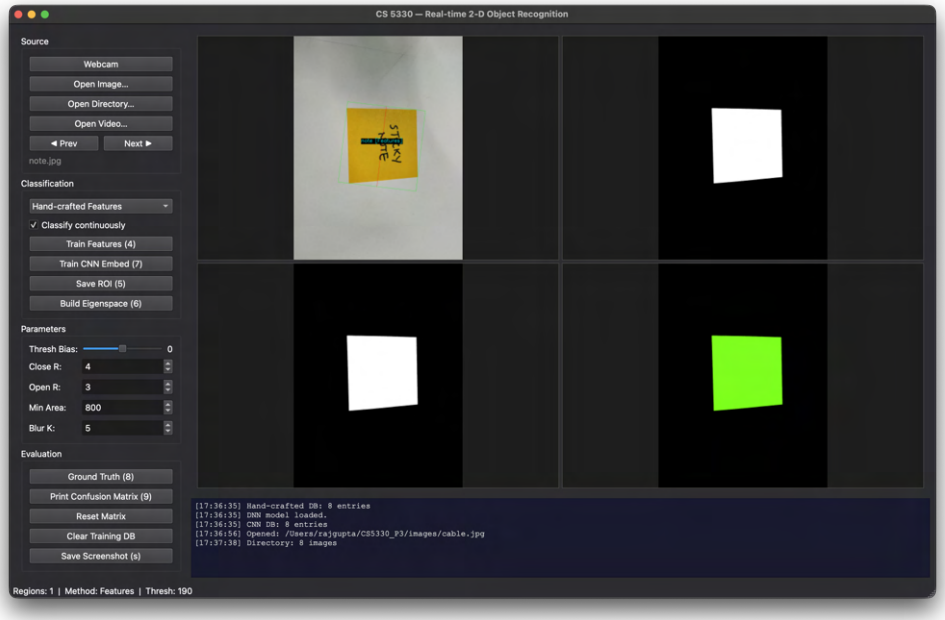



## 2. GUI - Source: Image

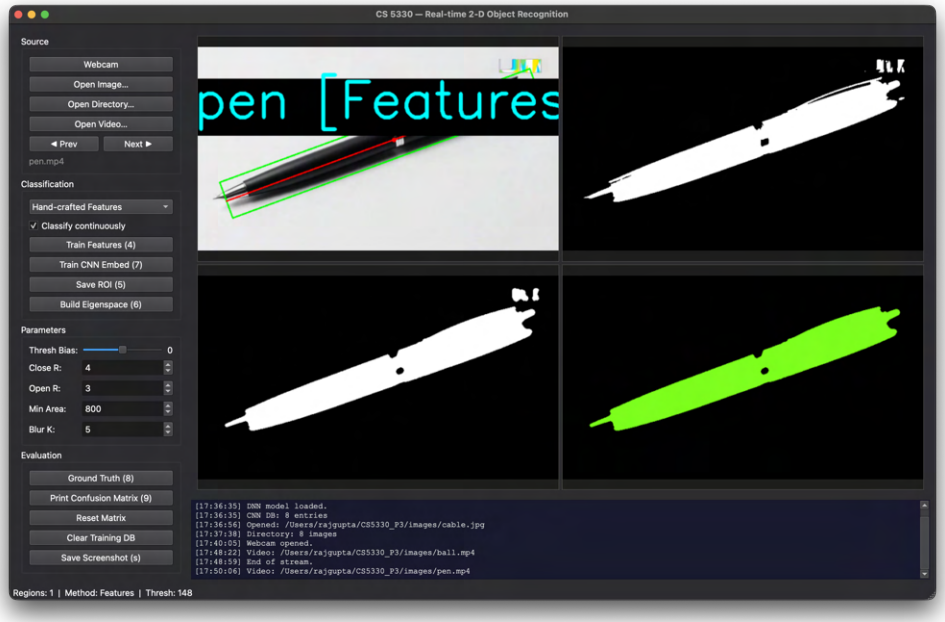


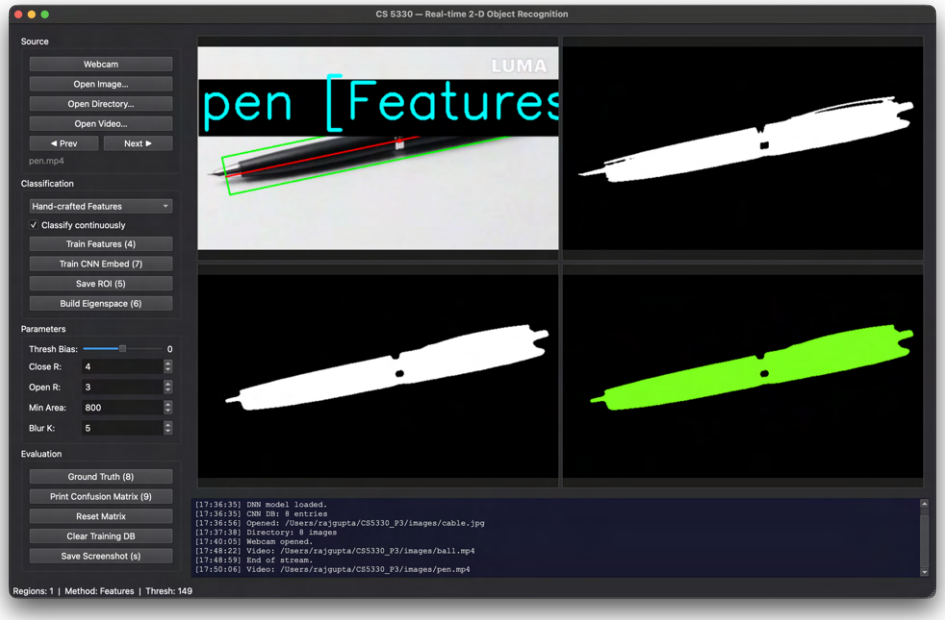

## 3. GUI - Source: Directory



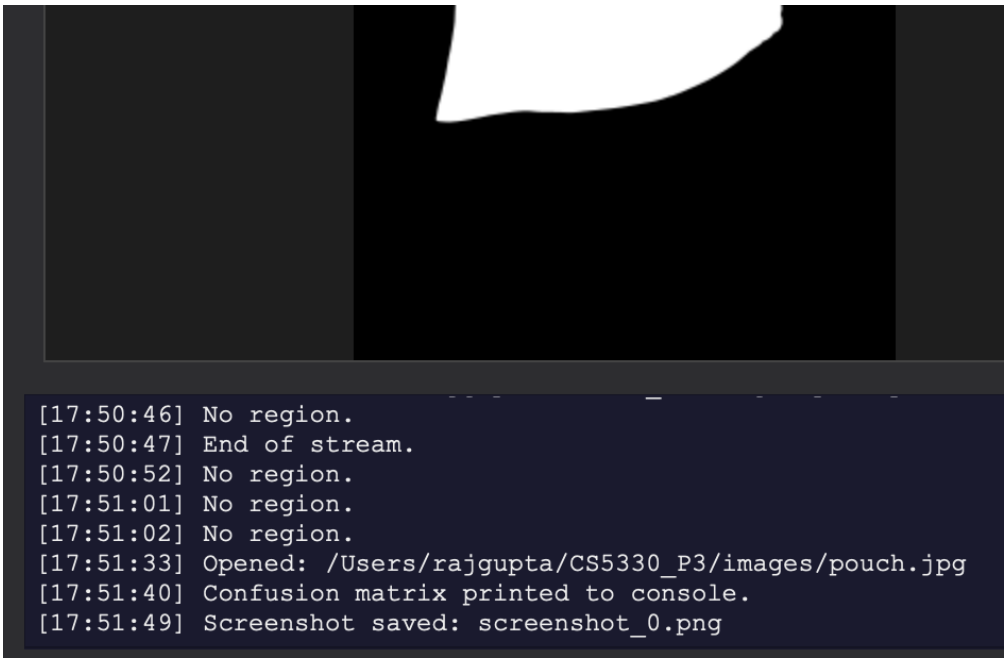




## 4. GUI - Source: Video




## 5. GUI - Save Screenshot and Print Confusion Matrix




## 2. All Four Pipeline Stages From Scratch

All four core stages of the pipeline were implemented from scratch without using any OpenCV built-in functions for these operations. Specifically: ISODATA adaptive thresholding with custom HSV-based scoring, erosion and dilation with circular structuring elements, two-pass connected components with union-find and 8-connectivity, and full moment computation including raw, central, normalised central moments, orientation, oriented bounding box, and all seven Hu moment invariants. OpenCV was used only for basic utilities like image I/O, colour space conversion, and Gaussian blur.

## 3. Both Embedding Methods Implemented (Eigenspace and CNN)

Both the eigenspace (PCA) and CNN (ResNet18) embedding methods were implemented for one-shot classification as described in Task 9, with comparative evaluation showing the strengths and weaknesses of each approach.

## Time Travel Days

3 days used.

## Reflection

This project was a thorough exercise in building a complete computer vision pipeline from the ground up. Implementing each stage from scratch, rather than calling a single OpenCV function, forced a deep understanding of what these algorithms actually do and why they work. For instance, writing the connected components algorithm made it clear why the two-pass approach with union-find is efficient, and implementing Hu moments revealed how higher-order moment invariants achieve rotation independence through careful combinations of normalized central moments.

The comparison between the three classification methods was the most interesting part. It was surprising that simple hand-crafted features (just 9 numbers) matched the CNN's accuracy on the training images, while the eigenspace with its pixel-level analysis performed so poorly at 37.5%. This reinforced the idea that choosing the right features matters more than having more features, and that data-driven methods like PCA

need sufficient training data to be useful. The confusion matrix evaluation further revealed that hand-crafted features, while strong at 87.5%, struggle with visually similar objects like the wallet and gloves that share similar fill ratios and aspect ratios.

Working with the thresholding also highlighted how much the rest of the pipeline depends on getting that first step right. Objects like the key, which has a light brass color, and the sticky note, which is bright yellow, required the HSV-based scoring approach to be captured properly. The key is detected through its moderate saturation, while the sticky note is captured primarily through its high saturation despite having a high value (brightness) that would make it hard to detect with simple intensity thresholding alone. When only part of an object was detected, the features changed dramatically, leading to misclassification. The choice of objects also mattered, having several predominantly dark objects with similar fill ratios and aspect ratios (wallet, pouch, gloves) in the set made classification more challenging and provided a realistic test of the system's limits.

## Acknowledgements


- **Professor Bruce Maxwell** and the course materials for CS5330, which provided the project specification, utilities.cpp, embedding.py, and the ResNet18 ONNX model.
- **OpenCV Documentation:** For references on image I/O, color space conversion, Gaussian blur, and the DNN module.
- **An AI assistant (Claude):** Was used to help write and debug code, implement the Qt GUI, and for project documentation.